Trace OS

Two-minute speaking script

Read this at a steady pace. It covers existing systems, the proposed pipeline, and what is implemented.

Existing systems

System monitors show CPU and memory at one instant, then forget them. A slow leak, a spawn storm, or rising disk use never becomes a history. Algorithm simulators run FCFS, paging, and deadlock examples correctly, but only on numbers typed in by hand. Learning models inside eBPF are fast, and they target networks and security. They do not name a scheduling or paging decision, and they do not replay a captured trace.

Proposed system

Trace OS is one pipeline on one trace. Observe a window of real signals. Detect the workload. Explain it with the operating-system concept behind it. Replay that same trace through classical algorithms, and write the comparison down. The aim is teaching. The demo needs no network and no language model.

Implementation

Observation uses psutil on Windows and on Linux. Once a second it records CPU, memory, process births, disk bytes, tasks actually executing, page faults, and context switches. One system snapshot counts those executing tasks, so the window does not open every process. These numbers are not the kernel run queue and not the page tables. Detection is a depth-limited decision tree trained on synthetic windows: normal, CPU saturation, memory growth, a spawn storm, or heavy I/O. One tree ships so the splits can be read aloud. Each detection has a confidence score, and an isolation forest marks a window unlike those five classes. The path is named CPU scheduling, memory and paging, process creation, or blocking I/O. Each window becomes a job and each sample a page id. FCFS, SJF, SRTF, priority, and round robin run beside FIFO, LRU, and optimal, on that same trace. The dashboard shows the Gantt chart and the fault counts. Frames and the round-robin quantum rerun the tables. The lab can peg the CPUs, climb memory, start a few processes, write a file, or collide two lock orders and give up. The report exports as Markdown or HTML.

Left out on purpose

eBPF, a local language model, Banker’s algorithm, and cgroups are not in this build. Random forests and XGBoost were proposed. The readable tree is what shipped, so the explanation stays the path the model actually took.